

Lab 1: CS527

Release date: 27 July 2026

Submission date: 3rd August 2026, 9AM.

Problem Statement:

Simulator a mini-computer system.

Users can write a program in a simple programming language, as specified in the language specification section. A compiler will compile the program, and will generate a byte code. Byte code will be read by a computer system, and will initialise a memory. The processor will read the instruction one by one to execute the program, and will modify the memory. At the end, memory will be written to the computer system in a file.

Finally, compilation and computer system should be invoked by a single main file, and executed as a single executable.

Language specification:

Language supports basic math operations : + - / *

Dest Variable = <Operand 1 variable> <operation> <Operand 2 variable>

Memory operations: Read and Write

Read <variable> <address>

Write <variable> <address> //address is a constant

Control operations: No control operation supported in today's lab.

Data movement operation:

<Dest Variable> = <constant value>

Variable names: x0 to x255.

Example: Add four variables

Read x1, 0

Read x2, 4

Read x3, 8

Read x4, 16
x5 = x1 + x2
x6 = x3 + x4
x7 = x5 + x6
Write x7, 20

Example output of the above program is:

5 1 0 0
5 2 4 0
5 3 8 0
5 4 12 0
1 5 1 2
1 6 3 4
1 7 5 6
6 7 16 0

Byte code specification

Compiler generates a byte code for every operation and finally the byte code is stored in file called "program.byte".

Format of each line of byte code

<operation> <dest register> <operand 1> <operand 2>

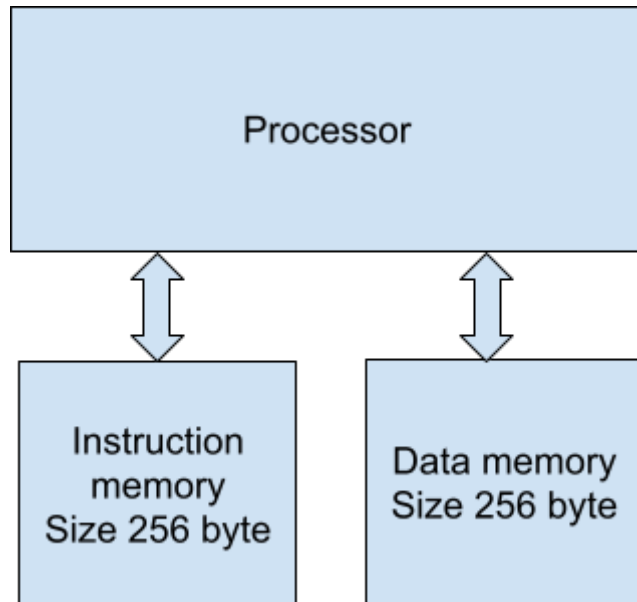
Operation	Opcode
Add	1
Subtract	2
Multiply	3
Divide	4
Memory read	5
Memory write	6
Data movement	7

In case of memory read, write and data movement operations - operand 2 is 0.

Note that

- Every line has four bytes
- The maximum value for each byte is 255.

Processor specification



Processor has four functions:

```
reset()
fetch()
decode()
execute()
```

Following data elements are there in processor:

```
Int Register[256]
Int PC, opcode, dest, src1, src2
extern char Instruction[256], Data[256];
Int end_of_simulation = 0
```

It has a global array of registers of size 256. And a global variable named PC

Reset() initializes each element of register array to 0 as well initialize PC to 0
fetch(): reads four bytes from the PC address of instruction memory and stores in opcode, dest, src1, and src2 respectively
decode() : void and empty function at this time
execute(): based on opcode, it performs the operations. Once see 0 opcode, set end_of_simulation = 1

Memory

Memory is coded as a separate 'c' file.

```
Char Instruction[256];  
Char Data[256];
```

Assumes program.byte and data.byte present in the directory. The memory is byte addressable which means when you read instructions from char array will read the instruction byte-by-byte.

```
initialize() {  
    //read the data from "program.byte" and populate Instruction memory  
    //read the data from "data.byte" and populate data memory.  
}  
  
finalize() {  
    //Write data.byte  
}
```

Main file

```
main() {  
  
    compile();  
    initialise();  
    reset();  
  
    while(end_of_simulation) {  
        fetch();  
        decode();  
        execute();  
    }  
  
    finalize();  
  
}
```

Directory structure

Following code files should be there:

main.c
compiler.c
compiler.h
processor.c
processor.h
memory.c
memory.h
Makefile

Some tutorials on multi-file compilation:

<https://www.codewithc.in/tutorials/multi-file-programs>

https://www.w3schools.com/c/c_organize_code.php

You can search more on the internet

Makefile tutorial:

<https://makefiletutorial.com/>

FAQs

Example user level programs:

1. Sum of N numbers where N is fixed at program time
2. Multiply two complex numbers
3. Find the determinant of a 3x3 matrix.

Following can be done when branch instructions are introduced.

4. Sum of N numbers where N is given at run time.
 - a. Four bytes starting from 0x0 stores N
 - b. Next 4N bytes store the N numbers.
5. FIR filter response:
 - a. Input start at 0x0 of data memory. First four bytes tells size of array, say N. Next 4N bytes are input array.
 - b. Output is stored at 0x10.
 - c. Weights are part of the program - as constants.

Data size

- In this assignment, data size is not specified. You can assume each operation is 8 bits, or 32 bits. 32 bit operation size is preferable.

Submission Instructions:

- Submit as a single zipped file - containing all your code, and test programs.